AICTE Application Validation (PS 26261): Problem and Solution

Oct 27, 2026 · @prathmesh

PS 26261 asks for an AI screening layer that helps AICTE officials validate approval applications. We build the checks, evidence and flagging, simulate the external verification, and leave every decision with the officials.

The official problem statement

The brief asks for an AI-enabled decision-support platform that checks AICTE approval applications at the earliest stage, while the competent authorities keep scrutiny, inspection and the final decision.

Background. AICTE (Ministry of Education) processes applications from technical institutions for Extension of Approval (EoA) and other approval-related changes, under the Approval Process Handbook, regulations and administrative instructions. Processing can involve verifying institutional records, scrutinising documents, checking compliance and, where applicable, further verification or inspection.

What the platform must do. Validate application information and supporting documents against available institutional records and configurable regulatory requirements; identify anomalies or document manipulation; facilitate verification through original issuing authorities; and flag possible infrastructure or other compliance gaps for review. It must provide evidence-linked findings and a clear verification status.

Objectives named in the brief:

- Identify missing, incomplete, expired or inconsistent information and documents at the application stage.
- Reduce repeated manual checking by using institutional records already in authorised AICTE systems.
- Detect potential document alterations, duplication and anomalies for further verification.
- Verify documents through issuing-authority portals, APIs, QR codes or digital signatures, wherever available and authorised.
- Check infrastructure-related evidence against configurable requirements and flag cases needing more evidence or inspection.

- Give transparent, traceable and explainable validation results.

Safeguards the brief makes essential:

- All checks and workflows are configurable to the applicable Handbook, regulations and instructions.
- Access to records and external sources is authorised and role-based; the solution must not assume access to restricted databases.
- Institutional and personal data is protected with security, confidentiality, access logging and retention safeguards.
- AI flags are explainable and evidence-supported.
- AI output is decision support only; statutory scrutiny, inspections and final decisions remain with the competent authorities.

Evaluation. The prototype is demonstrated on authorised, anonymised or synthetic data. The brief names these measures: document-extraction accuracy, validation accuracy, anomaly-detection performance, source-verification rate, discrepancy-identification reliability, average verification time, alert timeliness and reduction in manual effort.

The problem

AICTE officials check every approval application's documents by hand, and that scrutiny is slow, repetitive and inconsistent between officers.

AICTE is the national regulator for technical education. Colleges apply to renew or extend their approval (Extension of Approval, or EoA) and attach affidavits, NOCs, accreditation certificates, infrastructure photos, floor plans and fee payment proof.

Officials look for four kinds of trouble:

- **Missing or expired documents:** the NOC expired last year, or the affidavit is not attached.
- **Inconsistent information:** the college name on the NOC differs from the application, or the claimed intake differs from the last approved intake.
- **Possible tampering:** an altered date, a duplicated document, mismatched IDs.
- **Unverifiable documents:** confirming a certificate is genuine means contacting the issuing authority, which is slow.

The brief notes that the volume and variety of applications and supporting documents makes manual checking time-consuming, and that missing, expired or inconsistent items can delay processing and add administrative effort.

The solution

The solution is a first-pass screening layer in front of the human officers: it flags problems and orders the review queue, but never approves or rejects anything. The brief is explicit that AI output is decision-support only and final decisions stay with AICTE officials.

The flow for one application:

1. **Intake:** the college submits an application. The system identifies the category (for example, EoA renewal) and loads the matching checklist of required documents and rules.
2. **Read the documents:** OCR and a vision model classify each upload and extract structured fields such as names, dates, ID numbers and validity periods.
3. **Check completeness and validity:** the rules engine flags missing documents, expired certificates and empty required fields.
4. **Cross-check consistency:** extracted data is compared against other documents in the same application and against AICTE's existing institutional records (previous approvals, approved intake, deficiency history).
5. **Flag anomalies:** metadata and consistency checks surface suspicious signs, such as a creation date after the issue date, mismatched IDs or duplicate documents. Each flag shows its evidence and reason, and is labelled a preliminary indicator, never proof of forgery.
6. **Verify with the source:** where an issuing authority supports it (API, QR code or digital signature), the system checks the document and records the result as verified, mismatch, expired, pending, source unavailable or manual review required.
7. **Dashboard and audit trail:** officials see every application's status, open deficiencies and flags, with a log of which rules ran, which records were checked and what each source returned.
8. **Human decision:** officers review only the flagged cases and make the final call.

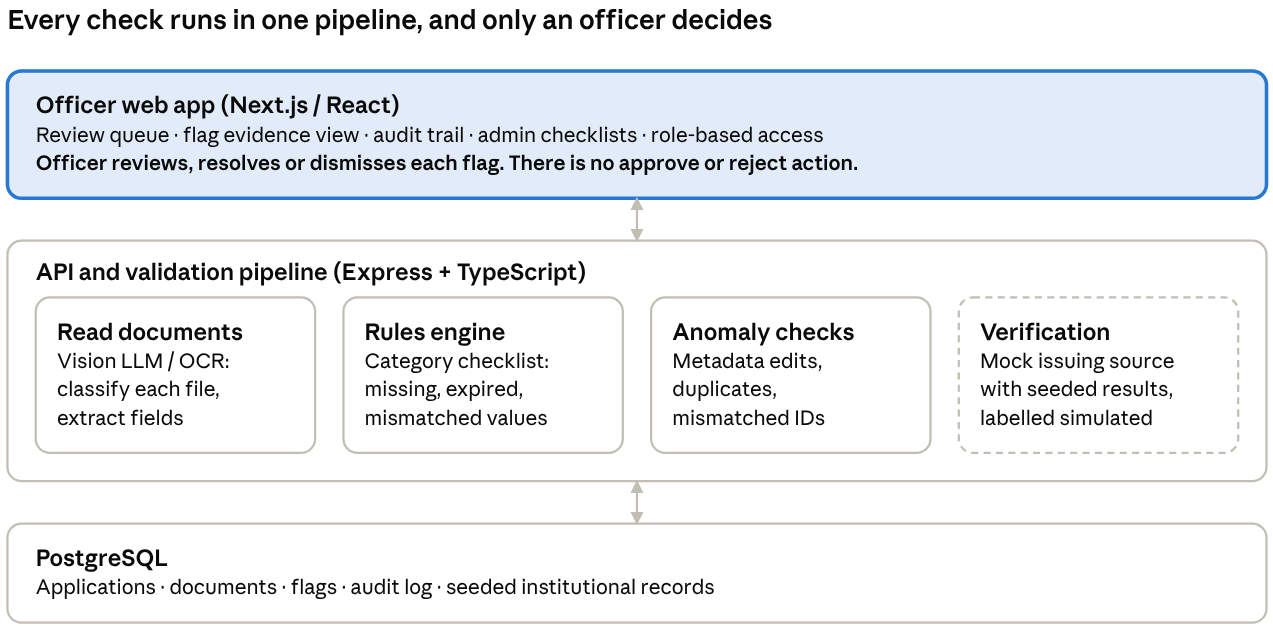
How the solution covers the brief

Every deliverable in the brief is addressed: six are built, two are built in part, one is simulated and one is designed for later integration. Status is stated plainly so the pitch matches what a judge sees running.

Brief deliverable	What we do	Status
Application-category checklists and configurable rules	A config table maps each application category to required documents and validity rules; an admin edits it in the app	Built
OCR classification, extraction and completeness checks	Vision-capable LLM or OCR reads each upload, identifies its type, extracts fields (names, certificate numbers, issue and expiry dates) and checks the result	Built
Cross-document and institutional-record consistency checks with evidence-linked flags	Compares fields across documents and against seeded institutional records (previous approval, approved intake); each flag shows found, expected and source	Built
Anomaly and potential-forgery indicators	Metadata checks (creation date after issue date, edited after signing), duplicate-document and mismatched-ID detection; each flag labelled a preliminary indicator	Built
Issuing-authority verification workflow	A mock verify action returns seeded results with source, timestamp and one of six statuses: verified, mismatch, expired, pending, source unavailable, manual review	Simulated
Compliance assistance for documentary and infrastructure evidence	Documentary checks run against the checklist; infrastructure photos get a light vision-LLM observation only, with trained floor-plan analysis out of scope	Partly built

Brief deliverable	What we do	Status
Dashboard with status, deficiencies, verification outcomes, pending actions, audit history	Review queue, application detail, verification panel and an append-only audit trail	Built
In-app alerts for missing information, expired documents, pending verification and suspected anomalies	An alert list on the dashboard fed by the same flags	Built, small
Role-based access, secure handling, human review before adverse action	Three roles (officer, senior officer, admin) and access actions logged in the audit trail; encryption and retention policy are designed, not built; no approve or reject action exists in the system	Partly built
Prototype integrated with, or designed for integration with, the AICTE portal	Stand-alone prototype on synthetic data, structured so it can plug into the portal's application data	Designed for

How it is built



architecture · 3 layers, 4 pipeline components

The officer app talks to one API; the pipeline reads, checks and flags each application, and everything it finds is stored with its evidence so an officer can inspect it. The dashed box marks the one simulated component.

How applications are prioritised

Priority is rule-based, so every position in the queue can be explained with a sentence an officer can check. It only orders the review queue; it never approves or rejects anything.

Review priority	Triggered by	Shown to the officer as
High	An expired mandatory certificate, or a major intake discrepancy	A red chip with the triggering rule in one line
Medium	Conflicting information that needs clarification	An amber chip with the conflicting fields named
Low	A minor missing field, or routine verification still pending	A green chip with what is outstanding

The system calls this "review priority", not "risk", because a risk label about an institution reads as an accusation, and the brief treats every AI flag as a preliminary indicator that a person must verify.

A worked example

Illustrative example: Institute X applies for renewal, and the dashboard shows three flags before an officer opens the file.

- The NOC expired in March.
- The application claims an intake of 240, while the last approved intake on record is 180.
- The affidavit's PDF was last edited after the date printed on it.

Each flag shows the exact field, the value found, the value expected and the reason. The officer opens the application already knowing where to look, instead of reading every page.

The pitch in one sentence

"We turn hours of manual document scrutiny into a prioritized, evidence-backed review queue, so officials spend their time on the applications that actually have problems, with every flag explainable and every final decision still theirs."

What is real and what is simulated

Most of the pipeline can be built for real; only external verification and infrastructure vision need to be simulated or scoped down, and the pitch should say so.

Part	Status in the build
Checklists and rules engine	Build for real
OCR, classification and field extraction (vision LLM)	Build for real
Cross-document and cross-record consistency checks	Build for real, against seeded institutional data
Metadata-based anomaly flags	Build for real
Dashboard and audit trail	Build for real
Issuing-authority verification	Simulate, and say so
Computer vision on infrastructure photos	Cut, or keep to a light vision-LLM observation

The brief itself says "wherever available and authorized", so simulating external verification is expected. Overclaiming forgery detection or real government API access is what would hurt the pitch.

Revised feature list

The revised build keeps the teammate's three core features, shrinks the dashboard, and adds the five deliverables the brief names that the original list lacked.

Feature	Origin	Build plan
Smart document intelligence (OCR, classification, field extraction)	Teammate	Core. Cite the page and quoted text; highlight exact text only if the OCR engine returns word bounding boxes. Confidence comes from OCR word scores or validation checks, not the model's own claim.
Cross-document contradiction detector	Teammate	Core and the demo centrepiece. Show found value, expected value, source document and discrepancy side by side; click a flag to inspect its evidence.
Review-priority queue (renamed from risk queue)	Teammate	Core. Rule-based high, medium and low with a stated reason; officers review, resolve or dismiss flags; never approves or rejects.
Command-centre dashboard	Teammate	Thin. Counts and charts only, with every figure labelled as demo data.
Issuing-authority verification	Added	Simulate. Mock verify button with seeded responses and status chips (verified, mismatch, expired, pending, source unavailable, manual review) plus timestamps.
Metadata anomaly flags	Added	Build. Creation date after issue date, edited after signing, duplicate documents; each labelled a preliminary indicator.
Configurable checklists by category	Added	Build. A config table mapping application category to required documents and rules, editable by an admin.
Audit trail	Added	Build. Append-only log of rules run, records checked and officer actions.
Role-based access	Added	Build, small. Officer, senior officer and admin roles.

Demo order: open an application with three flags, click the intake mismatch to show the evidence side by side, then show the queue ordering itself with a reason for each position.

Risks and how we reduce them

The biggest risk is overclaiming what the AI can detect, so the build leads with checks that can be explained.

Risk	How we reduce it
Overclaiming forgery detection	Lead with explainable metadata and consistency checks; label every flag a preliminary indicator requiring human verification
No real institutional or issuing-authority data	Seed realistic synthetic records, label them as synthetic, and demonstrate verification as simulated
Extraction accuracy on messy scans	Test early on varied skewed and low-quality documents; extractions that fail validation (a date that will not parse, an ID in the wrong format) go to manual review, never silent acceptance
Explainability added too late	Every flag stores the field, value found, value expected, source document and the rule that fired from the start
Scope creep	Make the core pipeline demo-stable first; simulated and scoped-down parts are not expanded until it is

Expected outcomes and how we will evaluate them

The outcomes below are the ones the built features support; each is stated as a design intent until it is measured.

- **Earlier identification of missing, expired or inconsistent information:** checklist, validity and cross-document checks run at the application stage.

- **Less repetitive manual scrutiny:** officers open a priority-ordered queue where each flag already shows its evidence.
- **Transparency, accountability and auditability:** evidence-linked flags and an append-only audit trail make every outcome traceable.
- **More consistent checking:** the same configurable checklist is applied to every application in a category.

We will evaluate against the measures the brief names, on a labelled synthetic test set: extraction accuracy, validation accuracy, anomaly-detection performance, discrepancy-identification reliability and alert timeliness. We have no measured figures yet, so none are claimed. Source-verification rate is not meaningful while verification is simulated, and we will say so.

Open items for the team

- Confirm who owns the OCR and extraction pipeline end to end; it decides whether the demo works on messy scans.
- Confirm the Theme and Problem Statement ID format for the submission.
- Decide whether to attempt word-level highlighting of flagged text (needs an OCR engine that returns bounding boxes) or settle for page-level citation with quoted text.
- Confirm the deployment choice; the current slide carries it over from the earlier deck.

Source

The text of Problem Statement 26261, as supplied by the team. Everything under "The official problem statement" is taken from it; the solution, build status and risk sections are our own design.